Document number: P1018r1
Audience: WG21

Ville Voutilainen
2018-06-09

Evolution status after Rapperswil 2018

Abstract

This paper is a collection of items Evolution has worked on up to and in the Rapperswil meeting, their status, and plans for the future.

Concepts

We discussed P0745R1 (Concepts in-place syntax) and P1079 (A minimal solution to the concepts syntax problems). While there was support for both approaches, we do not yet have an agreement on which approach to take.

Additionally, an adjustment was made for concept requirement, in how to define the resulting type of an expression. That is,

```
{ E } -> Concept<Args...>;
```

will use the exact type of E to check the Concept, as opposed to convertible types. Convertible types can still be checked by adding more constraints to the check.

We reviewed P0782R1, Constraining Concepts Overload Sets. This proposal changes dependent calls in constrained templates to consider only the overloads that were used to satisfy a constraint as viable candidates. Further work is expected, and we will revisit the proposal in San Diego, as we need to decide the fate of this change before C++20 ships.

Modules

No consensus yet to move a Modules subset forward. A merge of ATOM and the Modules TS will be applied to the Modules TS Working Draft.

Coroutines

We discussed P1063R0, Core Coroutines. Further work was encouraged, but we agreed to move the merge of the Coroutines TS forward.

Feature-testing macros

We approved the proposal to move feature-testing macros into the C++20 Working Draft.

Aggregate initialization

Aggregate initialization was discussed at length; we approved a change that makes class types with user-declared constructors non-aggregates. The proposal to allow paren-initializing aggregates did not reach consensus yet. We expect to do a holistic analysis of initialization.

Contracts

Contracts were approved, with the knowledge that there are still open questions and issues about preconditions and postconditions on virtual functions.

Generalized alias declarations

P0945R0, Generalizing alias declarations was discussed. We are not undoing any typename removals; the idea is to find a different syntax for generalized alias declarations; further work expected for San Diego.

Various constexpr extensions

- P1002R0, Try-catch blocks in constexpr functions was approved. A constexpr function can contain a try-catch, and constant evaluation is okay as long as an exception isn't thrown.
- P0784R3, More constexpr containers was approved. Constexpr allocations can now migrate to run-time.
- P1073R0, constexpr! functions was approved. This provides always-constexpr functions, which are paramount for value-based reflection.
- P0595R1, std::is_constant_evaluated() was approved and will go to LEWG next. This allows providing different code for constexpr evaluation and non-constexpr code.
- P1064R0, Allowing Virtual Function Calls in Constant Expressions was approved.
- P1077R0, Allowing Virtual Destructors to be "Trivial" was approved.
- P1045R0, constexpr Function Parameters was reviewed. Guidance was given to explore providing a new syntax for declaring function templates where there are no dependent types.

Template argument deduction

P1021R0, Extensions to Class Template Argument Deduction was reviewed. Further work is expected for San Diego, no approvals yet.

Deducing this

P0847R0, Deducing this was reviewed. The guidance is to go for providing a this-type identifier in the same place where cv-qualifiers and ref-qualifiers go, and to make using an identifier or using traditional this exclusive choices.

Unreviewed material

We had quite a pile of papers, and didn't get to all of them. There are copy elision papers, move relocation, a proposal to embed data in a program, a proposal to be able to extract an exception from a std::exception_ptr without rethrowing, and other such assorted stuff.

We have papers for tweaks for structured bindings, we expect to look at all of those in San Diego.